CE 4011 Term Project — Proposal

Ali Utku Tekin — 2744076 MSc. Structural Engineering, 3rd Semester · Spring 2025-2026 Date: 2026-05-16

1. Structural system and analysis method

The proposed program is a **2D structural analysis tool for plane frames and trusses**, implemented in Python and built on top of my Assignment 3 and Assignment 4 solver. It analyses any planar assembly of:

- **Frame members** — Euler-Bernoulli beam-columns with three DOFs per node (`ux`, `uy`, `rz`), optional moment releases (internal hinges), uniform distributed transverse loads, in-span point loads, and through-depth thermal gradients;
- **Truss members** — pin-pin axial bars with two DOFs per node and uniform axial temperature change.

The numerical engine is the standard **Direct Stiffness Method**. Per element, the 6×6 local stiffness `k_local` is rotated into global coordinates by `R^T · k_local · R`, element contributions are scattered into the global stiffness `K` and load vector `F` through the active-DOF map, and the partitioned system

```
K_ff · D_f = F_f − K_fs · D_s
```

is solved for the free displacements `D_f`, with `D_s` carrying the prescribed support settlements introduced in Assignment 4. Reactions are recovered from `R = K · D − F` on restrained rows, member end-forces from `q_local = k_local · d_local − p_local`, and an equilibrium-check pass sums `R^T · q_local` at every free node against the applied load to flag any residual above tolerance.

The solver carries the full Assignment 4 feature set: thermal axial and bending fixed-end forces from `FrameTemperatureLoad` and `TrussTemperatureLoad`, internal hinges via Schur static condensation, and support settlements through the partitioned solve above. These pieces are already implemented, tested, and validated against the textbook reference results for Assignment 4's Q2(a) and Q2(b).

2. Planned advanced feature — free-vibration modal analysis

The advanced feature is a **free-vibration eigenvalue solver** that augments the existing static program. For an undamped, unforced structure the equation of motion reduces to

```
(K − ω² · M) · φ = 0
```

a generalised eigenvalue problem in the active free-DOF block `(K_ff, M_ff)`. The structural stiffness `K` is already produced by the existing assembler; the new mass matrix `M` will be assembled using the **consistent-mass formulation** in the element's local frame and rotated to global coordinates with the same 6×6 transformation used for stiffness, so the mass and stiffness pipelines share their rotation and DOF-map machinery. Material density (`ρ`, kg/m³) becomes the only new physical property required from the input file.

The eigenproblem will be solved with `scipy.linalg.eigh` (symmetric, positive-definite mass), returning natural angular frequencies `ω_n` and mode shapes `φ_n`. The post-processor will convert these to frequencies `f_n = ω_n / (2π)` and periods `T_n = 1 / f_n`, normalise each mode (either mass-orthonormal `φ_n^T · M · φ_n = 1` or max-component = 1, user choice), and return them in a dedicated modal result object — see Section 5.

Required output of the modal feature:

1. A table of the lowest `n` natural frequencies and periods.
2. **Graphical mode-shape visualisation** in the GUI: for a selected mode index `k`, the deformed structure is plotted with displacements scaled by a user slider, overlaid on the undeformed geometry. This static plot is the required deliverable.

Time-domain animation of the mode shape (continuous sinusoidal sweep at `ω_n`) is treated as **optional polish**; it will be implemented only if time permits after the verification matrix in Section 5 is fully green. The fallback path is the static-overlay plot.

3. Input and output format

The current `.txt` solver input format is preserved for backward compatibility (all existing Assignment 3 and Assignment 4 fixtures load unchanged). A single addition is required for the modal feature:

```
MATERIALS n
<id> <E> <alpha> <density> [name]
```

The first four columns are positional and required in the new shape; the optional `name` follows. Legacy input files written before the modal feature may omit `density`, in which case it defaults to 0.0 kg/m³ on load; static analysis runs unchanged. **Modal analysis requires a positive density on every element's material** — the modal solver refuses to run on a model whose elements all carry `density = 0` and returns a clear error to the GUI.

Unit consistency. The static solver uses kN-m engineering units (`E` in kN/m², lengths in m, forces in kN), while material density is most naturally written in SI kg/m³. The modal module will convert density from kg/m³ into the consistent force-time-length system used by the static solver internally before assembling the mass matrix `M`, so the user supplies density in familiar SI units and the eigenvalue result comes out in s^{-1} (and Hz after dividing by 2π) without further unit work.

No new top-level block is required: modal analysis is a runtime mode chosen from the GUI's *Run → Modal* menu, not a property of the input file. The GUI also reads and writes its own `.spa.json` project files, which embed the canonical `.txt` model plus GUI-only state (labeled grid system, view limits, snap-target toggles).

Outputs:

- **Static run** — same console table and member-result format as Assignment 4 (free-DOF displacements, reactions, element end-forces, equilibrium residual).
- **Modal run** — frequency/period table per mode, plus the selected mode-shape rendered on the canvas.

4. Visualization tools (PyQt6 GUI)

The program exposes a single **PyQt6 + matplotlib graphical interface**. Modeling and result-presentation features already implemented and shipping on the cleanup commit are:

- **Modeling canvas** — node, frame, and truss placement; support and load application; an undo/redo command stack covering every model mutation.
- **Labeled grid system** — SAP2000-style named X- and Y-grid lines.
- **Multi-target snap engine** — node, grid intersection, element endpoint, element midpoint, and nearest-on-element snapping, with each snap target individually toggleable from the View menu.
- **Project I/O** — JSON project files (`.spa.json`) that round-trip the model, grid, view limits, and snap settings; plain `.txt` import/export remains available for solver compatibility.
- **Static result overlays** — deformed shape, bending-moment diagram, shear-force diagram, axial-force diagram, and reaction arrows, each toggleable.

New visualisation work for the term project:

- **Modal results pane** — a frequency-and-period table with a mode-index selector and a deformation-scale slider; selecting a mode redraws the canvas with that mode's deformed shape over the undeformed geometry.
- (Optional polish) Time-domain animation of the selected mode at its natural frequency, using matplotlib's `FuncAnimation` driving the Qt canvas.

5. Software architecture and verification

The class hierarchy stays exactly as drawn in the repository's existing UML (see `README.md`): `StructuralModel` aggregates `Node`, `Material`, `Section`, `Support`, `NodalLoad`, and the `Element2D` polymorphic hierarchy (`FrameElement2D` and `TrussElement2D`); a free-function `Assembler` builds `K` and `F`; the `Solver` runs the partitioned solution; and the `Postprocessor` computes member forces, reactions, and the equilibrium residual. The PyQt6 GUI sits on top through a thin controller layer and the existing command/undo stack.

The modal feature adds a small, isolated set of objects without disturbing this layout:

- `Material.density` — one new optional field.
- `MassAssembler` (module) — `assemble_mass_matrix(model, dofs)`, reusing the existing rotation and DOF-map utilities.
- `ModalAnalyzer` (module) — `solve_modal(K_ff, M_ff, n_modes)`, returning a dedicated **modal result object** with `frequencies`, `periods`, and `modes`. The static `AnalysisResult` is left untouched; the GUI dispatches on the result type rather than overloading a single class.
- `gui_qt/modal_view.py` and `gui_qt/dialogs.ModalDialog` — the table, slider, and *Run → Modal* entry point.

Verification plan (the matrix that must be green before the report is submitted):

#	Case	Pass criterion
1	Assignment 4 Q2(a) — settlement frame	Existing q2a regression test still matches the reference results.
2	Assignment 4 Q2(b) — thermal gradient	Existing q2b regression test still matches the reference results.
3	Simply-supported beam, point load (already in `tests/test_diagram_signs.py`)	`dM/dx = V` and analytical `PL/4` midspan moment.
4	Cantilever tip deflection `PL³/(3EI)` and rotation `PL²/(2EI)`	Existing closed-form regression tests still pass.
5	Clamped-free beam, first three natural frequencies	`β_n L = 1.875, 4.694, 7.855` → `f_n = (β_n L)² · √(EI / (ρ A L⁴)) / (2π)`, agreement to four significant figures.
6	Simply-supported beam natural frequencies	`f_n = (n π / L)² · √(EI / (ρ A)) / (2π)`.
7	Two-DOF shear-building textbook problem (Chopra, *Dynamics of Structures*)	Periods match the published reference.
8	Mode orthogonality	`φ_i^T · M · φ_j = 0` for `i ≠ j` within numerical tolerance.

All eight cases will be executed automatically through `pytest`. Cases 1-4 already pass on the current branch (93 / 93 tests at the time of this proposal). Cases 5-8 are added together with the modal feature in the implementation stage.

6. Schedule

Window	Deliverable
2026-05-17 → 2026-05-19	Finalise this proposal; submit.
2026-05-21 → 2026-06-05	Implement the mass assembler, the modal solver, and the modal results pane; cases 5-8 of the verification matrix turn green.
2026-06-06 → 2026-06-14	User manual, install manual, verification document, final class-diagram update, project report.
2026-06-15	Live demonstration.
2026-06-16 → 2026-06-19	Final polish; upload the `Report.zip` deliverable.

Repository: `https://github.com/tekcentu/Assignment4_FINAL` (current working branch: `claude/add-solver-gui-pyqt`).